

# **Competitive Programming**

**Section 35 Editorial: Math Mixed Contest**

Competitive Programming Course

2026-08-11

# Table of contents

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| <b>1</b>  | <b>Category coverage</b>                              | <b>5</b>  |
| <b>2</b>  | <b>Editorial overview</b>                             | <b>6</b>  |
| 2.1       | Read this derivation before the code . . . . .        | 6         |
| <b>3</b>  | <b>A. Blocked Path</b>                                | <b>8</b>  |
| 3.1       | Algorithm . . . . .                                   | 8         |
| 3.2       | Correctness proof . . . . .                           | 8         |
| 3.3       | Full C++ solution . . . . .                           | 8         |
| 3.4       | Full Python solution . . . . .                        | 9         |
| <b>4</b>  | <b>B. Coprime Pairs</b>                               | <b>11</b> |
| 4.1       | Algorithm . . . . .                                   | 11        |
| 4.2       | Correctness proof . . . . .                           | 11        |
| 4.3       | Full Python solution . . . . .                        | 11        |
| 4.4       | Full C++ solution . . . . .                           | 12        |
| <b>5</b>  | <b>C. At Least One</b>                                | <b>14</b> |
| 5.1       | Algorithm . . . . .                                   | 14        |
| 5.2       | Correctness proof . . . . .                           | 14        |
| 5.3       | Full C++ solution . . . . .                           | 14        |
| 5.4       | Full Python solution . . . . .                        | 15        |
| <b>6</b>  | <b>D. Complete Alphabet</b>                           | <b>16</b> |
| 6.1       | Restatement and constraints . . . . .                 | 16        |
| 6.2       | Observation and worked trace . . . . .                | 16        |
| 6.3       | Algorithm . . . . .                                   | 16        |
| 6.4       | Correctness proof . . . . .                           | 17        |
| 6.5       | Complexity, implementation notes, and tests . . . . . | 17        |
| 6.6       | Full C++ solution . . . . .                           | 17        |
| 6.7       | Full Python solution . . . . .                        | 18        |
| 6.8       | D-specific stress testing . . . . .                   | 19        |
| <b>7</b>  | <b>Testing discussion</b>                             | <b>20</b> |
| <b>8</b>  | <b>What to carry forward</b>                          | <b>21</b> |
| <b>9</b>  | <b>Problem-local upsolve checklist</b>                | <b>22</b> |
| 9.1       | A. Blocked Path . . . . .                             | 22        |
| 9.2       | B. Coprime Pairs . . . . .                            | 22        |
| 9.3       | C. At Least One . . . . .                             | 23        |
| <b>10</b> | <b>E. Modular Appointment</b>                         | <b>24</b> |
| 10.1      | Restatement and reduction . . . . .                   | 24        |
| 10.2      | Solvability . . . . .                                 | 24        |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| 10.3      | Algorithm . . . . .                                                | 24        |
| 10.4      | Correctness proof . . . . .                                        | 25        |
| 10.5      | Complexity and integer safety . . . . .                            | 25        |
| 10.6      | Common wrong answers . . . . .                                     | 25        |
| 10.7      | Tests that matter . . . . .                                        | 25        |
| 10.8      | Full C++ solution . . . . .                                        | 25        |
| 10.9      | Full Python solution . . . . .                                     | 26        |
| <b>11</b> | <b>F. Festival Synchronization</b>                                 | <b>27</b> |
| 11.1      | Model and solvability . . . . .                                    | 27        |
| 11.2      | Correctness proof . . . . .                                        | 27        |
| 11.3      | Complexity and tests . . . . .                                     | 27        |
| 11.4      | Full C++ solution . . . . .                                        | 28        |
| 11.5      | Full Python solution . . . . .                                     | 29        |
| <b>12</b> | <b>G. Capped Allocation</b>                                        | <b>30</b> |
| 12.1      | From unrestricted allocations to violations . . . . .              | 30        |
| 12.2      | Correctness proof . . . . .                                        | 30        |
| 12.3      | Complexity and tests . . . . .                                     | 30        |
| 12.4      | Full C++ solution . . . . .                                        | 31        |
| 12.5      | Full Python solution . . . . .                                     | 32        |
| <b>13</b> | <b>H. GCD-One Subsets</b>                                          | <b>34</b> |
| 13.1      | Count divisible subsets before exact gcds . . . . .                | 34        |
| 13.2      | Correctness proof . . . . .                                        | 34        |
| 13.3      | Complexity and tests . . . . .                                     | 34        |
| 13.4      | Full C++ solution . . . . .                                        | 34        |
| 13.5      | Full Python solution . . . . .                                     | 35        |
| <b>14</b> | <b>I. Exactly K Colors</b>                                         | <b>37</b> |
| 14.1      | Separate which colors from how they are used . . . . .             | 37        |
| 14.2      | Correctness proof . . . . .                                        | 37        |
| 14.3      | Complexity and tests . . . . .                                     | 37        |
| 14.4      | Full C++ solution . . . . .                                        | 37        |
| 14.5      | Full Python solution . . . . .                                     | 39        |
| <b>15</b> | <b>J. Target LCM Subsets</b>                                       | <b>41</b> |
| 15.1      | Compress each value to the target exponents it completes . . . . . | 41        |
| 15.2      | Correctness proof . . . . .                                        | 41        |
| 15.3      | Complexity and tests . . . . .                                     | 41        |
| 15.4      | Full C++ solution . . . . .                                        | 42        |
| 15.5      | Full Python solution . . . . .                                     | 43        |
| <b>16</b> | <b>K. Weighted Coupon Collection</b>                               | <b>45</b> |
| 16.1      | Tail probabilities turn completion time into a sum . . . . .       | 45        |
| 16.2      | Correctness proof . . . . .                                        | 45        |
| 16.3      | Complexity and tests . . . . .                                     | 45        |
| 16.4      | Full C++ solution . . . . .                                        | 46        |
| 16.5      | Full Python solution . . . . .                                     | 47        |
| <b>17</b> | <b>L. Random Divisor Descent</b>                                   | <b>48</b> |
| 17.1      | Expectation recurrence . . . . .                                   | 48        |
| 17.2      | Correctness proof . . . . .                                        | 48        |

|                                     |    |
|-------------------------------------|----|
| 17.3 Complexity and tests . . . . . | 48 |
| 17.4 Full C++ solution . . . . .    | 48 |
| 17.5 Full Python solution . . . . . | 50 |

# 1 Category coverage

| Category                            | Coverage problem    |
|-------------------------------------|---------------------|
| modular arithmetic                  | A, C, F, G, I, K, L |
| GCD, LCM, and Diophantine equations | E, F                |
| sieve, primes, and factorization    | B, H, J, L          |
| combinatorics under modulo          | A, G, I             |
| inclusion-exclusion                 | B, D, G, H, I, K    |
| probability and expected value      | C, K, L             |

Modular Appointment supplies the Euclidean/congruence reasoning that Coprime Pairs does not exercise directly. Its complete local package is included by this section's checker.

## 2 Editorial overview

### 2.1 Read this derivation before the code

The detailed upsolve material for each problem is collected in **Problem-local upsolve checklist** below the compact reference listings. Read the corresponding block before studying its implementation:

| Problem | State/model to establish                             | Trace to perform                   | Main failure mode                                    |
|---------|------------------------------------------------------|------------------------------------|------------------------------------------------------|
| A       | all monotone paths minus paths through one block     | 3 by 4, block (2,2)                | applying the formula to multiple blocks              |
| B       | <code>seen[d]</code> for square-free divisors        | process 1, 6, 10, 21, 30           | updating <code>seen</code> before counting           |
| C       | product of no-event probabilities                    | 1/2, 1/3, 1/4                      | multiplying without independence                     |
| D       | strings that omit a chosen set of symbols            | length 4, alphabet size 3          | adding overlapping omissions without correction      |
| E       | one linear congruence after moving the offset        | a solvable and impossible gcd case | dividing before the gcd test                         |
| F       | merge two residue classes                            | noncoprime compatible moduli       | using ordinary CRT blindly                           |
| G       | unrestricted allocations minus cap violations        | two small capped boxes             | forgetting the <code>cap+1</code> shift              |
| H       | subsets divisible by <code>d</code> , then exact gcd | values 2,3,4                       | subtracting in the wrong order                       |
| I       | choose colors, then force all to appear              | length 3, two selected colors      | counting strings using at most <code>k</code> colors |
| J       | prime-exponent requirements as bits                  | target 12                          | keeping values that do not divide the target         |
| K       | sum tail probabilities                               | two unequal coupon weights         | treating coupon arrivals as independent              |
| L       | expectation from proper divisors                     | values 4 and 6                     | including the current value as a divisor choice      |

The source listings are intentionally compact reference implementations. The checklist supplies the constraints, pseudocode, proof checkpoints, complexity, language notes, adversarial tests,

and brute-force oracle to use before code.

| Problem                       | Main tools                                       |
|-------------------------------|--------------------------------------------------|
| A. Blocked Path               | combinations, subtract bad paths                 |
| B. Coprime Pairs              | SPF, distinct prime factors, inclusion-exclusion |
| C. At Least One               | complement probability                           |
| D. Complete Alphabet          | inclusion-exclusion, modular exponentiation      |
| E. Modular Appointment        | linear congruence, extended Euclid               |
| F. Festival Synchronization   | generalized CRT                                  |
| G. Capped Allocation          | stars and bars, inclusion-exclusion              |
| H. GCD-One Subsets            | divisor sieve, exact-gcd inversion               |
| I. Exactly K Colors           | combinations, surjections                        |
| J. Target LCM Subsets         | factorization, bitmask DP                        |
| K. Weighted Coupon Collection | expectation, inclusion-exclusion                 |
| L. Random Divisor Descent     | expectation DP, divisor sieve                    |

## 3 A. Blocked Path

### 3.1 Algorithm

```
total = C(H+W-2, H-1)
bad = C(R+C-2, R-1) * C(H-R+W-C, H-R)
answer = total - bad
```

### 3.2 Correctness proof

Every monotone path is determined by the positions of its down moves. The bad paths are exactly those that can be split into a monotone path from the start to the blocked cell and then from the blocked cell to the finish. These two parts are independent, so multiplication counts them. Subtracting all bad paths leaves exactly the valid paths.  $\square$

### 3.3 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long r = 1;
    while (b) {
        if (b & 1LL) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int h, w, r, c;
    if (!(cin >> h >> w >> r >> c)) return 0;
    int limit = h + w;
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
```



```

for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
auto C = [&](int n, int k) -> long long {
    if (k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n - k] % MOD;
};
long long total = C(h + w - 2, h - 1);
long long through = C(r + c - 2, r - 1) * C(h - r + w - c, h - r) % MOD;
cout << (total - through + MOD) % MOD << '\n';
return 0;
}

```

### 3.4 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(n: int) -> tuple[list[int], list[int]]:
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (n + 1)
    invfact[n] = pow(fact[n], MOD - 2, MOD)
    for i in range(n, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def comb(n: int, k: int, fact: list[int], invfact: list[int]) -> int:
    if k < 0 or k > n:
        return 0
    return fact[n] * invfact[k] % MOD * invfact[n - k] % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    h, w, r, c = data
    fact, invfact = prepare(h + w)
    total = comb(h + w - 2, h - 1, fact, invfact)
    through = (
        comb(r + c - 2, r - 1, fact, invfact)
        * comb(h - r + w - c, h - r, fact, invfact)
        % MOD
    )

```

```
    print((total - through) % MOD)

if __name__ == "__main__":
    main()
```

## 4 B. Coprime Pairs

### 4.1 Algorithm

Process numbers left to right. For current  $x$ , get its distinct prime factors. Use inclusion-exclusion over nonempty subsets to count previous numbers sharing at least one factor. Add  $\text{previous\_count} - \text{shared}$  to the answer, then update all subset-product counters for  $x$ .

### 4.2 Correctness proof

For current  $x$ , a previous number is non-coprime with  $x$  iff it is divisible by at least one distinct prime factor of  $x$ . The subset-product counters give the size of every intersection of these divisibility events. Inclusion-exclusion therefore gives the number of previous non-coprime values. All other previous values form coprime pairs with  $x$ . Each pair is counted when its second endpoint is processed.  $\square$

### 4.3 Full Python solution

```
import sys
from collections import defaultdict

def spf_table(n: int) -> list[int]:
    spf = list(range(n + 1))
    if n >= 1:
        spf[1] = 1
    for p in range(2, n + 1):
        if spf[p] == p and p * p <= n:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    arr = data[1 : 1 + n]
    spf = spf_table(max(arr))
    seen = defaultdict(int)
    answer = 0
```

```

for idx, x in enumerate(arr):
    factors = []
    y = x
    while y > 1:
        p = spf[y]
        factors.append(p)
        while y % p == 0:
            y //= p
    share = 0
    for mask in range(1, 1 << len(factors)):
        prod = 1
        bits = 0
        for i, p in enumerate(factors):
            if (mask >> i) & 1:
                prod *= p
                bits += 1
        share += seen[prod] if bits % 2 else -seen[prod]
    answer += idx - share
    for mask in range(1, 1 << len(factors)):
        prod = 1
        for i, p in enumerate(factors):
            if (mask >> i) & 1:
                prod *= p
        seen[prod] += 1
print(answer)

```

## 4.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<int> a(n);
    int mx = 1;
    for (int &x : a) {
        cin >> x;
        mx = max(mx, x);
    }

    vector<int> spf(mx + 1);
    iota(spf.begin(), spf.end(), 0);
    if (mx >= 1) spf[1] = 1;
    for (long long p = 2; p <= mx; p++) {

```

```

        if (spf[p] == p && p * p <= mx) {
            for (long long x = p * p; x <= mx; x += p) {
                if (spf[x] == x) spf[x] = p;
            }
        }
    }

    vector<int> seen(mx + 1, 0);
    long long answer = 0;
    for (int idx = 0; idx < n; idx++) {
        int x = a[idx];
        vector<int> factors;
        while (x > 1) {
            int p = spf[x];
            factors.push_back(p);
            while (x % p == 0) x /= p;
        }

        long long share = 0;
        int m = (int)factors.size();
        for (int mask = 1; mask < (1 << m); mask++) {
            int prod = 1, bits = 0;
            for (int i = 0; i < m; i++) {
                if (mask & (1 << i)) {
                    prod *= factors[i];
                    bits++;
                }
            }
            if (bits % 2) share += seen[prod];
            else share -= seen[prod];
        }
        answer += idx - share;

        for (int mask = 1; mask < (1 << m); mask++) {
            int prod = 1;
            for (int i = 0; i < m; i++) {
                if (mask & (1 << i)) prod *= factors[i];
            }
            seen[prod]++;
        }
    }

    cout << answer << '\n';
    return 0;
}

```

## 5 C. At Least One

### 5.1 Algorithm

Compute the probability that no event happens, then subtract from 1:

```
none = product((q_i-p_i)/q_i)
answer = 1 - none
```

### 5.2 Correctness proof

The complement of “at least one event happens” is “no event happens.” Events are independent, so the probability that none happen is the product of the individual failure probabilities. Complementing gives the desired probability.  $\square$

### 5.3 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long r = 1;
    while (b) {
        if (b & 1LL) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    long long none = 1;
    for (int i = 0; i < n; i++) {
        long long p, q;
        cin >> p >> q;
        none = none * ((q - p) % MOD) % MOD * mod_pow(q, MOD - 2) % MOD;
    }
}
```

```

}
cout << (1 - none + MOD) % MOD << '\n';
return 0;
}

```

## 5.4 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    idx = 1
    none = 1
    for _ in range(n):
        p, q = data[idx], data[idx + 1]
        idx += 2
        none = none * ((q - p) % MOD) % MOD * pow(q, MOD - 2, MOD) % MOD
    print((1 - none) % MOD)

if __name__ == "__main__":
    main()

```

## 6 D. Complete Alphabet

### 6.1 Restatement and constraints

There are  $k$  symbols, numbered from 1 to  $k$ . Count the strings of length  $n$  in which every symbol appears at least once, modulo 1,000,000,007. Here  $1 \leq n \leq 10^{18}$  and  $1 \leq k \leq 20$ . The huge length rules out building strings or dynamic programming over positions; the small alphabet is the important bound.

This is a *surjection* count: each of the  $k$  symbols must receive at least one of the  $n$  positions. It is not simply  $k^n$ , because that includes strings that miss one or more symbols.

### 6.2 Observation and worked trace

For each symbol  $s$ , let  $E_s$  be the set of strings that never use  $s$ . We want strings outside the union of these bad sets. If we choose a particular set of  $j$  missing symbols, every position has  $k-j$  choices, so exactly  $(k-j)^n$  strings miss all selected symbols. There are  $C(k, j)$  choices for that selected set.

For  $n=4$ ,  $k=3$ , begin with all  $3^4=81$  strings. Subtract the strings missing one named symbol:  $3 * 2^4 = 48$ . A string using only one symbol was subtracted twice, so add back  $C(3, 2) * 1^4 = 3$ . Missing all three symbols contributes  $0^4=0$  because  $n$  is positive. The answer is:

$$\begin{aligned} 3^4 - C(3, 1)2^4 + C(3, 2)1^4 - C(3, 3)0^4 \\ = 81 - 48 + 3 - 0 \\ = 36 \end{aligned}$$

The sign alternates because a string missing exactly  $r$  symbols belongs to  $C(r, j)$  selected missing sets at level  $j$ . Its total coefficient is  $\sum_j (-1)^j C(r, j) = (1-1)^r$ : zero when  $r>0$ , and one when  $r=0$ .

### 6.3 Algorithm

```
answer = 0
for missing from 0 to k:
    term = C(k, missing) * (k - missing)^n modulo MOD
    if missing is even:
        answer += term
    else:
        answer -= term
print answer modulo MOD
```



Compute every power with binary exponentiation. Since  $k \leq 20$ , ordinary integer binomial coefficients fit comfortably before reduction; both languages can compute them directly.

## 6.4 Correctness proof

For a fixed selected set  $S$  of missing symbols, a string avoids every symbol in  $S$  exactly when all its positions use one of the other  $k - \text{missing}$  symbols. Thus it contributes  $(k - \text{missing})^n$  strings. Summing over the  $C(k, \text{missing})$  possible  $S$  counts each string with the usual inclusion-exclusion sign.

Consider any string that misses exactly  $r$  symbols. It occurs once for every selected subset of those  $r$  missed symbols, so its coefficient in the sum is  $\sum_{j=0}^r (-1)^j C(r, j)$ . This is 1 for  $r=0$  and 0 for  $r>0$ . Therefore every string using all symbols is counted once and every invalid string is cancelled, which is precisely the required answer. QED.

## 6.5 Complexity, implementation notes, and tests

There are  $k+1$  terms, and each modular power takes  $O(\log n)$ , for total time  $O(k \log n)$  and  $O(1)$  extra space. Keep the answer reduced after every addition/subtraction. In C++, use `long long` for modular products; in Python, use `pow(base, n, MOD)` rather than constructing `base**n`.

Test  $n < k$  (answer zero),  $k=1$  (answer one),  $n=k$ , and a very large  $n$  to make sure a linear-in- $n$  solution cannot slip through. A common error is to subtract  $k \cdot (k-1)^n$  without restoring strings that miss two symbols. Another is treating  $0^0$  casually; the local constraint  $n \geq 1$  makes the final term zero when every symbol is selected as missing.

## 6.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1'000'000'007LL;

long long mod_pow(long long base, long long exponent) {
    long long result = 1;
    while (exponent > 0) {
        if (exponent & 1LL) result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1LL;
    }
    return result;
}

long long choose(int n, int r) {
    r = min(r, n - r);
    long long result = 1;
```

```

    for (int i = 1; i <= r; i++) result = result * (n - r + i) / i;
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    int k;
    if (!(cin >> n >> k)) return 0;

    long long answer = 0;
    for (int missing = 0; missing <= k; missing++) {
        long long term = choose(k, missing) * mod_pow(k - missing, n) % MOD;
        if (missing % 2 == 0) answer += term;
        else answer -= term;
        answer %= MOD;
    }
    cout << (answer + MOD) % MOD << '\n';
    return 0;
}

```

## 6.7 Full Python solution

```

import math
import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, k = data

    answer = 0
    for missing in range(k + 1):
        term = math.comb(k, missing) * pow(k - missing, n, MOD)
        if missing % 2 == 0:
            answer += term
        else:
            answer -= term
    print(answer % MOD)

```

```
if __name__ == "__main__":  
    main()
```

## 6.8 D-specific stress testing

For tiny cases ( $n \leq 6$ ,  $k \leq 4$ ), enumerate all  $k^n$  strings directly and count those whose set of used symbols has size  $k$ . This oracle never uses the inclusion-exclusion formula, so it catches sign, combination, and overlap mistakes independently. Include  $n < k$ , where no string can use every symbol, and  $k = 1$ , where the one possible string is complete.

The separate scale case uses  $k = 20$  and an  $n$  near  $10^{18}$ . It verifies that the implementation uses logarithmic modular exponentiation for each term, rather than attempting a linear-in- $n$  enumeration, DP, or ordinary power.

## 7 Testing discussion

The mixed contest generators compare against direct combinatorics, brute-force gcd pair checks, and exact rational probability. This phase is especially prone to off-by-one and complement mistakes, so stress tests are part of the workflow.

## 8 What to carry forward

Math contest problems are often a composition of small tools. The main skill is recognizing the composition early: combinations plus subtraction, sieve plus inclusion-exclusion, or probability plus complement. # Extended upsolve workshop

**A. Blocked Path.** Subtract  $\text{ways}(\text{start}, \text{block}) * \text{ways}(\text{block}, \text{finish})$  from all monotone paths. Test a blocked start or finish, where every path is forbidden.

**B. Coprime Pairs.** For each value, inclusion-exclusion over its distinct prime factors counts earlier values sharing any factor; subtract from prior count. Test value one and repeated composite values.

**C. At Least One.** Compute  $1 - \text{product}(1 - p_i/q_i)$ . Directly adding event probabilities double-counts overlapping outcomes; test one certain event. # Full upsolve notes

**A. Blocked Path.** Every forbidden monotone path has a unique prefix ending at  $(R, C)$  and suffix from it, so subtract  $C(R+C-2, R-1) * C(H-R+W-C, H-R)$  from all paths. For a 3-by-3 grid blocked in the middle:  $6 - 2 * 2 = 2$ . Test a blocked start/end, where the answer is zero.

**B. Coprime Pairs.** Maintain  $\text{seen}[d]$ , the number of prior values divisible by square-free  $d$ . For 30 after  $[6, 10, 21]$ , inclusion-exclusion gives  $\text{seen}[2] + \text{seen}[3] + \text{seen}[5] - \text{seen}[6] - \text{seen}[10] - \text{seen}[15] + \text{seen}[30]$  all earlier values share a factor. Count before updating the subset products. SPF plus at most 127 subsets per local value gives safe complexity. Test one, duplicates, and a three-factor value.

**C. At Least One.** Compute  $\text{none} = \text{product}((q-p) * \text{inverse}(q))$ , then answer  $1 - \text{none}$ . For  $1/2$  and  $1/3$ , this is  $1 - 1/2 * 2/3 = 2/3$ . Independence justifies the product; a certain event and a zero-probability event are essential tests.

**D. Complete Alphabet.** Let a bad event be that one named symbol is absent. For  $n=4, k=3$ , inclusion-exclusion is  $3^4 - 3 * 2^4 + 3 * 1^4 = 36$ . Test  $n < k$ , one symbol, and a huge exponent; the alternating signs repair all overlaps between absent-symbol events.

## 9 Problem-local upsolve checklist

### 9.1 A. Blocked Path

#### 9.1.1 Constraints and model

$H$  and  $W$  are up to one million, so grid DP over  $H \times W$  cells is not viable. There is exactly one blocked cell. A monotone path is completely determined by the positions of its down moves, which makes binomial coefficients the correct model. Precompute factorials through  $H+W$  once.

#### 9.1.2 Trace and pseudocode

For  $H=3, W=4, R=2, C=2$ , all paths are  $C(5,2)=10$ . The blocked prefix count is  $C(2,1)=2$ ; the suffix count is  $C(3,1)=3$ ; subtract 6 to get 4.

```
all = choose(H+W-2, H-1)
through = choose(R+C-2, R-1) * choose(H-R+W-C, H-R)
answer = all - through modulo MOD
```

Every forbidden path has one and only one such prefix/suffix pair, proving the multiplication and subtraction. The time is  $O(H+W)$  preprocessing and  $O(1)$  work afterward. In C++ use `long long` products; Python uses the same factorial lists. Wrong answers usually forget that a blocked start or end makes the answer zero, or apply this one-block formula to multiple blocks. Test both endpoints, a one-row grid, and a block adjacent to an endpoint.

### 9.2 B. Coprime Pairs

#### 9.2.1 Constraints and state

The array has up to 200,000 values, each at most one million. Build SPF once. For each square-free divisor  $d$ , `seen[d]` is the number of already processed values divisible by  $d$ . Prime powers are deliberately absent: sharing a prime is the event being counted.

#### 9.2.2 Trace and pseudocode

When processing  $30=2 \cdot 3 \cdot 5$  after  $[6, 10, 21]$ , count shared values as:

```
+seen[2] + seen[3] + seen[5]
-seen[6] - seen[10] - seen[15]
+seen[30]
```

This is 3, hence it contributes  $3-3=0$  coprime pairs. Then increment every one of those seven seen entries for future values.

```

factors = distinct prime factors of x from SPF
shared = alternating sum seen[product(subset)]
answer += processed - shared
for every nonempty subset: seen[product(subset)] += 1

```

An earlier value is included in the alternating sum once iff it shares at least one prime with  $x$ ; otherwise it appears in no selected event. Therefore `processed-shared` is precisely the number of new coprime pairs. With at most seven distinct factors, each value creates at most 127 subsets. Complexity is  $O(M \log \log M + \sum 2^{\omega(a[i])})$  time and  $O(M)$  memory. Test 1 (no subsets, coprime with all prior values), repeated composites, and primes.

### 9.2.3 Implementation and failure modes

Generate subset products from distinct factors, not from the factorization with multiplicity. Count before updating `seen`. Use `long long` for the answer: there can be about  $n*(n-1)/2$  pairs. A brute-force oracle for random arrays of length at most 10 should compare every gcd directly.

## 9.3 C. At Least One

### 9.3.1 Constraints and model

The statement explicitly gives independent events and prime-modulus rational probabilities with nonzero denominators. Directly adding event probabilities overcounts outcomes where several events occur. The complement has one product: none of the events happens.

### 9.3.2 Trace, proof, and code shape

For probabilities  $1/2$ ,  $1/3$ ,  $1/4$ , the no-event probability is  $(1/2)*(2/3)*(3/4)=1/4$ , so the answer is  $3/4$ . In modular arithmetic compute each factor as  $(q-p) * \text{inverse}(q)$ , multiply them, and normalize  $1-\text{none}$ .

```

none = 1
for p, q:
    none = none * (q-p) * inverse(q) modulo MOD
print (1-none) modulo MOD

```

Independence proves the product; complementarity proves the final subtraction. The algorithm takes  $O(n \log \text{MOD})$  with Fermat inverses and  $O(1)$  extra memory. Test one event, a zero-probability event, and a certain event. Do not use this method if events are dependent or a denominator is zero modulo MOD.

## 10 E. Modular Appointment

### 10.1 Restatement and reduction

For each query, find the smallest  $x \geq 0$  such that:

$$a \cdot x + c \equiv b \pmod{m}$$

Move the constant term to the other side:

$$a \cdot x \equiv b - c \pmod{m}$$

This is a linear congruence, not always a modular-division problem. Writing  $\text{target} = b - c$ , the congruence is equivalent to the Diophantine equation:

$$a \cdot x + m \cdot y = \text{target}$$

### 10.2 Solvability

Let  $g = \gcd(a, m)$ . Every value  $a \cdot x + m \cdot y$  is divisible by  $g$ , so  $g \mid \text{target}$  is necessary. Bézout's identity gives integers  $p, q$  with  $a \cdot p + m \cdot q = g$ ; multiplying by  $\text{target}/g$  proves the same condition is sufficient.

If the condition holds, divide the congruence by  $g$ :

$$(a/g) \cdot x \equiv \text{target}/g \pmod{m/g}$$

Now  $a/g$  and  $m/g$  are coprime, so  $a/g$  has an inverse modulo  $m/g$ .

### 10.3 Algorithm

For each query:

1. Normalize  $\text{target} = (b - c) \bmod m$  into  $[0, m)$ .
2. Use extended Euclid on  $(a, m)$  to obtain  $g$  and a coefficient  $p$  such that  $a \cdot p + m \cdot q = g$ .
3. If  $\text{target} \% g \neq 0$ , print -1.
4. Otherwise compute:

$$\begin{aligned} \text{modulus} &= m/g \\ x &= (\text{target}/g) * p \bmod \text{modulus} \end{aligned}$$

5. Normalize  $x$  into  $[0, \text{modulus})$ . This is the smallest nonnegative member of the solution class.



## 10.4 Correctness proof

If `target` is not divisible by `g`, the Diophantine equation has no solution, so printing `-1` is correct.

Otherwise extended Euclid gives  $a \cdot p + m \cdot q = g$ . Multiplying by `target/g` and reducing modulo `m` gives:

$$a * (p * \text{target}/g) \equiv \text{target} \pmod{m}$$

Thus the computed residue is a solution. After division by `g`, the coefficient `a/g` is invertible modulo `m/g`, so there is exactly one solution class modulo `m/g`. Normalizing that class into  $[0, m/g)$  selects its unique smallest nonnegative representative. Therefore every query output is correct.

## 10.5 Complexity and integer safety

Extended Euclid takes  $O(\log \min(a, m))$  time and  $O(\log \min(a, m))$  recursive stack space per query. In C++, multiplying the Bézout coefficient by `target/g` may overflow 64 bits even though the normalized answer does not; the reference uses `__int128` for that product.

## 10.6 Common wrong answers

- Taking an inverse of `a` modulo `m` before checking  $\gcd(a, m) = 1$ .
- Checking  $g \mid b$  instead of  $g \mid (b-c)$ .
- Forgetting that C++ `%` may leave a negative residue.
- Returning an arbitrary integer solution instead of the smallest nonnegative one.
- Mishandling  $m/g = 1$ ; the unique normalized answer is zero.

## 10.7 Tests that matter

```
target = 0           -> x = 0
gcd(a,m) = 1         -> ordinary modular inverse
gcd(a,m) > 1 and divides target
gcd(a,m) > 1 and does not divide target -> -1
b < c               -> negative unnormalized target
m/g = 1             -> answer 0
Values near 10^9    -> multiplication safety
```

For small moduli, the oracle can simply try every `x` from 0 to `m-1`.

## 10.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;using ll=long long;
ll eg(ll a,ll b,ll&x,ll&y){if(!b){x=1;y=0;return a;}ll x1,y1,g=eg(b,a%b,x1,y1);x=y1;y=x1-a/b;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int q;cin>>q;while(q--){ll a,b,c,m;c
```

## 10.9 Full Python solution

```
import math,sys
def eg(a,b):
    if b==0:return (a,1,0)
    g,x,y=eg(b,a%b);return g,y,x-a//b*y
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));q=d[0];i=1;out=[]
    for _ in range(q):
        a,b,c,m=d[i:i+4];i+=4;t=(b-c)%m;g,x,_=eg(a,m)
        out.append("-1" if t%g else str((t//g*x)%(m//g)))
    print("\n".join(out))
if __name__=="__main__":main()
```

# 11 F. Festival Synchronization

## 11.1 Model and solvability

Write a candidate common time as  $x = r_1 + m_1 \cdot k$ . Substituting this into the second congruence gives one linear congruence for  $k$ :

$$m_1 \cdot k \equiv r_2 - r_1 \pmod{m_2}.$$

Let  $g = \gcd(m_1, m_2)$ . The congruence is solvable exactly when  $g$  divides  $r_2 - r_1$ . This is the compatibility condition for noncoprime CRT: the two requested residues must agree modulo the gcd of their moduli.

When compatible, divide by  $g$ :

$$(m_1/g) \cdot k \equiv (r_2 - r_1)/g \pmod{m_2/g}.$$

The new coefficient and modulus are coprime, so extended Euclid supplies the inverse of  $m_1/g$  modulo  $m_2/g$ . Normalize  $k$  into that modulus, then compute  $x = r_1 + m_1 \cdot k$ . All common solutions differ by  $\text{lcm}(m_1, m_2)$ .

## 11.2 Correctness proof

If the gcd does not divide the residue difference, every value  $m_1 \cdot k - m_2 \cdot t$  is divisible by the gcd and can never equal that difference, so  $-1$  is necessary. Otherwise the reduced inverse constructs a  $k$  satisfying the second congruence, while the representation  $r_1 + m_1 \cdot k$  satisfies the first by construction. Changing  $k$  by  $m_2/g$  changes  $x$  by  $m_1 \cdot m_2/g$ , the lcm. Thus normalizing gives the unique common solution in  $[0, \text{lcm})$ , which is the smallest nonnegative one.

## 11.3 Complexity and tests

Each query takes  $O(\log \min(m_1, m_2))$  time and logarithmic recursion depth. Use 64-bit storage for the lcm and a widened product for coefficient multiplication.

```
coprime moduli; equal moduli; compatible noncoprime moduli
residues disagree modulo the gcd -> impossible
one modulus is 1; answer is already r1
moduli near 10^9 -> product safety
```

## 11.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;

int64 extended_gcd(int64 a, int64 b, int64& x, int64& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int64 x1, y1;
    int64 g = extended_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    while (q--) {
        int64 r1, m1, r2, m2;
        cin >> r1 >> m1 >> r2 >> m2;

        int64 inverse, unused;
        int64 g = extended_gcd(m1, m2, inverse, unused);
        int64 difference = r2 - r1;
        if (difference % g != 0) {
            cout << -1 << '\n';
            continue;
        }

        int64 reduced_modulus = m2 / g;
        int64 multiplier = static_cast<int64>(
            (__int128)(difference / g) * inverse % reduced_modulus
        );
        if (multiplier < 0)
            multiplier += reduced_modulus;

        int64 period = m1 / g * m2;
        int64 answer = static_cast<int64>(
            ((__int128)m1 * multiplier + r1) % period
        );
        cout << answer << '\n';
    }
}
```

```
}  
}
```

## 11.5 Full Python solution

```
import sys  
  
def extended_gcd(a: int, b: int) -> tuple[int, int, int]:  
    if b == 0:  
        return a, 1, 0  
    g, x1, y1 = extended_gcd(b, a % b)  
    return g, y1, x1 - (a // b) * y1  
  
def main() -> None:  
    data = list(map(int, sys.stdin.buffer.read().split()))  
    q = data[0]  
    output = []  
    index = 1  
  
    for _ in range(q):  
        r1, m1, r2, m2 = data[index:index + 4]  
        index += 4  
        g, inverse, _ = extended_gcd(m1, m2)  
        difference = r2 - r1  
        if difference % g:  
            output.append("-1")  
            continue  
  
        reduced_modulus = m2 // g  
        multiplier = (difference // g * inverse) % reduced_modulus  
        period = m1 // g * m2  
        answer = (r1 + m1 * multiplier) % period  
        output.append(str(answer))  
  
    print("\n".join(output))  
  
if __name__ == "__main__":  
    main()
```

## 12 G. Capped Allocation

### 12.1 From unrestricted allocations to violations

Without upper bounds, the number of nonnegative solutions of  $x_1 + \dots + x_n = S$  is the stars-and-bars count  $C(S+n-1, n-1)$ . To enforce caps, define a bad event  $E_i$  meaning  $x_i \geq \text{cap}[i] + 1$ .

For a chosen subset  $T$  of violated boxes, subtract  $\text{cap}[i] + 1$  tokens from every  $i$  in  $T$ . The remaining sum is:

```
remaining = S - sum over i in T of (cap[i]+1).
```

If it is nonnegative, the intersection of those violations contains  $C(\text{remaining}+n-1, n-1)$  allocations. Inclusion-exclusion therefore gives:

```
sum over all subsets T:  
  (-1)^|T| * C(S-sum(cap[i]+1)+n-1, n-1).
```

The lower argument is at most 19, so each combination can be computed by a short multiplicative product. The stated bounds keep the upper argument below the modulus, avoiding Lucas-theorem complications.

### 12.2 Correctness proof

Stars and bars counts every unrestricted allocation once. For any allocation, let  $V$  be exactly the set of caps it violates. Its total coefficient in the formula is  $\sum_{T \subset V} (-1)^{|T|}$ , which is one when  $V$  is empty and zero otherwise. Hence every valid allocation remains once and every invalid allocation cancels. The variable shift is a bijection between an intersection of violations and the corresponding unrestricted smaller-sum problem.

### 12.3 Complexity and tests

There are  $2^n$  subsets and  $O(n)$  multiplication per combination, giving  $O(n \cdot 2^n)$  time and  $O(2^n)$  memory for subset sums.

```
S=0; one box; all caps zero  
S greater than the sum of all caps -> 0  
caps so large that no violation is possible  
the exact boundary  $x_i = \text{cap}[i] + 1$  -> verifies the +1 shift
```

## 12.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
const int64 MOD = 1'000'000'007;

int64 power(int64 base, int64 exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1)
            result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}

int64 choose_small_k(int64 top, int k, int64 inverse_factorial) {
    if (top < k || top < 0)
        return 0;
    int64 numerator = 1;
    for (int i = 1; i <= k; ++i) {
        numerator = numerator * ((top - i + 1) % MOD) % MOD;
    }
    return numerator * inverse_factorial % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    int64 target;
    cin >> n >> target;
    vector<int64> cap(n);
    for (int64& value : cap)
        cin >> value;

    int subset_count = 1 << n;
    int64 factorial = 1;
    for (int i = 1; i < n; ++i)
        factorial = factorial * i % MOD;
    int64 inverse_factorial = power(factorial, MOD - 2);
    vector<int64> shifted_sum(subset_count);
    int64 answer = 0;
    for (int mask = 0; mask < subset_count; ++mask) {
        if (mask) {
            int bit = __builtin_ctz(mask);
```

```

        shifted_sum[mask] = shifted_sum[mask ^ (1 << bit)] + cap[bit] + 1;
    }
    int64 remaining = target - shifted_sum[mask];
    int64 ways = choose_small_k(
        remaining + n - 1, n - 1, inverse_factorial
    );
    if (__builtin_popcount(static_cast<unsigned>(mask)) & 1)
        answer = (answer - ways + MOD) % MOD;
    else
        answer = (answer + ways) % MOD;
}

cout << answer << '\n';
}

```

## 12.5 Full Python solution

```

import sys

MOD = 1_000_000_007

def choose_small_k(top: int, k: int, inverse_factorial: int) -> int:
    if top < k or top < 0:
        return 0
    numerator = 1
    for i in range(1, k + 1):
        numerator = numerator * (top - i + 1) % MOD
    return numerator * inverse_factorial % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, target = data[:2]
    cap = data[2:]

    subset_count = 1 << n
    factorial = 1
    for i in range(1, n):
        factorial = factorial * i % MOD
    inverse_factorial = pow(factorial, MOD - 2, MOD)
    shifted_sum = [0] * subset_count
    answer = 0
    for mask in range(subset_count):
        if mask:
            least_bit = mask & -mask
            bit = least_bit.bit_length() - 1

```



```

        shifted_sum[mask] = shifted_sum[mask ^ least_bit] + cap[bit] + 1

    remaining = target - shifted_sum[mask]
    ways = choose_small_k(
        remaining + n - 1,
        n - 1,
        inverse_factorial,
    )
    answer += -ways if mask.bit_count() & 1 else ways

print(answer % MOD)

if __name__ == "__main__":
    main()

```

## 13 H. GCD-One Subsets

### 13.1 Count divisible subsets before exact gcds

For each divisor  $d$ , count how many input values are divisible by  $d$ ; call this `count[d]`. Every nonempty subset made only from those values has gcd divisible by  $d$ , so there are:

```
all_divisible[d] = 2^count[d] - 1
```

These subsets split into disjoint classes by their exact gcd. Consequently:

```
all_divisible[d] = exact[d] + exact[2d] + exact[3d] + ...
```

Process  $d$  downward. Every strict multiple already has its exact count, so:

```
exact[d] = all_divisible[d] - sum exact[multiple]
```

The answer is `exact[1]`.

### 13.2 Correctness proof

A subset contributes to `all_divisible[d]` precisely when every selected value is divisible by  $d$ , equivalently when its gcd is a multiple of  $d$ . Exact gcd classes are disjoint and cover all nonempty subsets, proving the multiple-sum identity. Downward induction then shows that subtracting the already correct strict-multiple classes leaves exactly the subsets with gcd  $d$ . In particular, the final state for  $d=1$  is the requested count.

### 13.3 Complexity and tests

Both the frequency-over-multiples pass and the exact-gcd subtraction perform a harmonic number of iterations:  $O(M \log M)$  time and  $O(M+n)$  memory, where  $M=\max(a)$ .

```
one value equal to 1; no pair with gcd 1
many duplicate values -> indices must remain distinct choices
all values share a factor -> answer 0
gcd 1 appears only after selecting three or more values
```

### 13.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

const int MOD = 1'000'000'007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> values(n);
    int maximum = 0;
    for (int& value : values) {
        cin >> value;
        maximum = max(maximum, value);
    }

    vector<int> frequency(maximum + 1);
    for (int value : values)
        ++frequency[value];

    vector<int> powers(n + 1, 1);
    for (int i = 1; i <= n; ++i)
        powers[i] = 2LL * powers[i - 1] % MOD;

    vector<int> exact(maximum + 1);
    for (int divisor = maximum; divisor >= 1; --divisor) {
        int divisible_count = 0;
        long long remove = 0;
        for (int multiple = divisor; multiple <= maximum; multiple += divisor) {
            divisible_count += frequency[multiple];
            if (multiple != divisor)
                remove += exact[multiple];
        }
        exact[divisor] = (powers[divisible_count] - 1 - remove) % MOD;
        if (exact[divisor] < 0)
            exact[divisor] += MOD;
    }

    cout << exact[1] << '\n';
}

```

## 13.5 Full Python solution

```
import sys
```

```

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    values = data[1:]
    maximum = max(values)

    frequency = [0] * (maximum + 1)
    for value in values:
        frequency[value] += 1

    powers = [1] * (n + 1)
    for i in range(1, n + 1):
        powers[i] = 2 * powers[i - 1] % MOD

    exact = [0] * (maximum + 1)
    for divisor in range(maximum, 0, -1):
        divisible_count = 0
        remove = 0
        for multiple in range(divisor, maximum + 1, divisor):
            divisible_count += frequency[multiple]
            if multiple != divisor:
                remove += exact[multiple]
        exact[divisor] = (powers[divisible_count] - 1 - remove) % MOD

    print(exact[1])

if __name__ == "__main__":
    main()

```

## 14 I. Exactly K Colors

### 14.1 Separate which colors from how they are used

First choose the  $k$  colors that appear:  $C(m,k)$  choices. For one fixed set of  $k$  colors, count length- $n$  strings in which every chosen color appears. There are  $k^n$  unrestricted strings, but some omit colors.

If a specified set of missing colors is absent, every position has  $k - \text{missing}$  choices. Inclusion-exclusion gives the number of surjections:

```
onto(n,k) = sum from missing=0 to k of
    (-1)^missing * C(k,missing) * (k-missing)^n.
```

The answer is  $C(m,k) * \text{onto}(n,k)$ . Precompute factorials and inverse factorials through the largest  $m$ ; use binary exponentiation because  $n$  may be  $10^{18}$ .

### 14.2 Correctness proof

Every valid string determines exactly one set of  $k$  used colors, so choosing the set creates disjoint cases. Within a fixed set, inclusion-exclusion keeps a string exactly when its missing-color set is empty: a string missing  $r$  chosen colors receives coefficient  $\sum C(r,j) (-1)^j$ , equal to zero for  $r > 0$  and one for  $r = 0$ . Thus the formula counts every string using exactly  $k$  colors once and no other string.

### 14.3 Complexity and tests

Precomputation costs  $O(\max m)$ . A query costs  $O(k \log n)$ , and the promised sum of  $k$  bounds the total work. Memory is  $O(\max m)$ .

```
k=1; k=m; k>n -> 0
n=1; a huge n testing binary exponentiation
distinguish exactly k from at most k
```

### 14.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
const int64 MOD = 1'000'000'007;

int64 power(int64 base, int64 exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1)
            result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    vector<tuple<int64, int, int>> queries(q);
    int maximum_m = 0;
    for (auto& [n, m, k] : queries) {
        cin >> n >> m >> k;
        maximum_m = max(maximum_m, m);
    }

    vector<int64> factorial(maximum_m + 1, 1);
    vector<int64> inverse_factorial(maximum_m + 1, 1);
    for (int i = 1; i <= maximum_m; ++i)
        factorial[i] = factorial[i - 1] * i % MOD;
    inverse_factorial[maximum_m] = power(factorial[maximum_m], MOD - 2);
    for (int i = maximum_m; i >= 1; --i)
        inverse_factorial[i - 1] = inverse_factorial[i] * i % MOD;

    auto choose = [&](int n, int k) {
        return factorial[n] * inverse_factorial[k] % MOD
            * inverse_factorial[n - k] % MOD;
    };

    for (auto [length, colors, used] : queries) {
        int64 onto = 0;
        for (int missing = 0; missing <= used; ++missing) {
            int64 term = choose(used, missing)
                * power(used - missing, length) % MOD;
            if (missing & 1)
                onto = (onto - term + MOD) % MOD;
            else

```

```

        onto = (onto + term) % MOD;
    }
    cout << choose(colors, used) * onto % MOD << '\n';
}
}

```

## 14.5 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    q = data[0]
    queries = [tuple(data[i:i + 3]) for i in range(1, 1 + 3 * q, 3)]
    maximum_m = max(m for _, m, _ in queries)

    factorial = [1] * (maximum_m + 1)
    for i in range(1, maximum_m + 1):
        factorial[i] = factorial[i - 1] * i % MOD
    inverse_factorial = [1] * (maximum_m + 1)
    inverse_factorial[-1] = pow(factorial[-1], MOD - 2, MOD)
    for i in range(maximum_m, 0, -1):
        inverse_factorial[i - 1] = inverse_factorial[i] * i % MOD

    def choose(n: int, k: int) -> int:
        return (
            factorial[n]
            * inverse_factorial[k]
            * inverse_factorial[n - k]
            % MOD
        )

    output = []
    for length, colors, used in queries:
        onto = 0
        for missing in range(used + 1):
            term = (
                choose(used, missing)
                * pow(used - missing, length, MOD)
                % MOD
            )
            onto += -term if missing & 1 else term
        answer = choose(colors, used) * (onto % MOD) % MOD
        output.append(str(answer))

```

```
print("\n".join(output))

if __name__ == "__main__":
    main()
```



## 15 J. Target LCM Subsets

### 15.1 Compress each value to the target exponents it completes

Any selected value must divide  $T$ ; otherwise the subset lcm contains an excess prime or exponent and can never return to  $T$ . Factor:

$$T = p_1^{e_1} * p_2^{e_2} * \dots * p_r^{e_r}.$$

For a value dividing  $T$ , set bit  $i$  when its exponent of  $p_i$  is exactly  $e_i$ . Lower exponents do not complete that prime's target requirement. The lcm of a subset is  $T$  exactly when the bitwise OR of its masks is the full mask.

Many indices may share a mask. If a mask occurs  $c$  times, there are  $2^c - 1$  ways to choose a nonempty group of those indices, and every such choice has the same effect on the OR. Merge each mask type into a bitmask DP:

```
next = dp                                # choose none of this type
next[old OR type] += dp[old] * (2count[type] - 1)
```

For  $T=1$ , the full mask is zero, so subtract the empty subset explicitly.

### 15.2 Correctness proof

Filtering is safe because lcm exponents are maxima and can never decrease. For retained values, the OR mask records exactly which target exponents the subset maximum has reached. Grouping equal masks partitions index choices into “none” and one of all nonempty selections, preserving their exact multiplicity. By induction over mask types,  $dp[mask]$  counts precisely the subsets producing that OR. The full OR is equivalent to every target prime reaching its required exponent, hence to lcm  $T$ .

### 15.3 Complexity and tests

Factoring  $T$  costs  $O(\sqrt{T})$ . At most nine distinct primes fit into a number up to  $10^9$ , so the DP costs  $O(4^r)$  in the loose bound of all type and state pairs, at most about 262000 operations, plus input processing.

```
T=1 with several ones; values not dividing T
one value already equals T; several indices with the same mask
every prime requirement supplied by a different value
correct primes but insufficient target exponents
```

## 15.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1'000'000'007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, target;
    cin >> n >> target;
    vector<int> values(n);
    for (int& value : values)
        cin >> value;

    vector<pair<int, int>> factors;
    int remaining = target;
    for (int prime = 2; 1LL * prime * prime <= remaining; ++prime) {
        if (remaining % prime)
            continue;
        int exponent = 0;
        while (remaining % prime == 0) {
            remaining /= prime;
            ++exponent;
        }
        factors.push_back({prime, exponent});
    }
    if (remaining > 1)
        factors.push_back({remaining, 1});

    int mask_count = 1 << factors.size();
    vector<int> frequency(mask_count);
    for (int value : values) {
        if (target % value)
            continue;
        int mask = 0;
        int copy = value;
        for (int bit = 0; bit < static_cast<int>(factors.size()); ++bit) {
            auto [prime, target_exponent] = factors[bit];
            int exponent = 0;
            while (copy % prime == 0) {
                copy /= prime;
                ++exponent;
            }
            if (exponent == target_exponent)
                mask |= 1 << bit;
        }
        ++frequency[mask];
    }
```

```

}

vector<int> dp(mask_count);
dp[0] = 1;
for (int type = 0; type < mask_count; ++type) {
    if (!frequency[type])
        continue;
    int nonempty_ways = 1;
    for (int i = 0; i < frequency[type]; ++i)
        nonempty_ways = 2LL * nonempty_ways % MOD;
    nonempty_ways = (nonempty_ways - 1 + MOD) % MOD;

    vector<int> next = dp;
    for (int mask = 0; mask < mask_count; ++mask) {
        int combined = mask | type;
        next[combined] = (
            next[combined] + 1LL * dp[mask] * nonempty_ways
        ) % MOD;
    }
    dp.swap(next);
}

int full_mask = mask_count - 1;
int answer = dp[full_mask];
if (full_mask == 0)
    answer = (answer - 1 + MOD) % MOD;
cout << answer << '\n';
}

```

## 15.5 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, target = data[:2]
    values = data[2:2 + n]

    factors = []
    remaining = target
    prime = 2
    while prime * prime <= remaining:
        if remaining % prime == 0:
            exponent = 0

```

```

        while remaining % prime == 0:
            remaining //= prime
            exponent += 1
        factors.append((prime, exponent))
    prime += 1
if remaining > 1:
    factors.append((remaining, 1))

mask_count = 1 << len(factors)
frequency = [0] * mask_count
for value in values:
    if target % value:
        continue
    mask = 0
    copy = value
    for bit, (prime, target_exponent) in enumerate(factors):
        exponent = 0
        while copy % prime == 0:
            copy //= prime
            exponent += 1
        if exponent == target_exponent:
            mask |= 1 << bit
    frequency[mask] += 1

dp = [0] * mask_count
dp[0] = 1
for type_mask, count in enumerate(frequency):
    if count == 0:
        continue
    nonempty_ways = (pow(2, count, MOD) - 1) % MOD
    next_dp = dp.copy()
    for mask, ways in enumerate(dp):
        combined = mask | type_mask
        next_dp[combined] += ways * nonempty_ways
        next_dp[combined] %= MOD
    dp = next_dp

full_mask = mask_count - 1
answer = dp[full_mask]
if full_mask == 0:
    answer -= 1
print(answer % MOD)

if __name__ == "__main__":
    main()

```

# 16 K. Weighted Coupon Collection

## 16.1 Tail probabilities turn completion time into a sum

Let  $T$  be the number of draws needed. For a nonnegative integer-valued random variable:

$$E[T] = \sum_{t \geq 0} P(T > t).$$

After  $t$  draws,  $T > t$  means at least one coupon type is still missing. Apply inclusion-exclusion to these missing events. For a nonempty set  $S$  of coupon types, all types in  $S$  are missing exactly when every draw avoids them. If  $\text{weight}(S)$  is their total weight and  $W$  is the total weight, that probability is:

$$(1 - \text{weight}(S)/W)^t.$$

Summing this geometric series over  $t \geq 0$  gives  $W/\text{weight}(S)$ . Therefore:

$$E[T] = \sum_{\text{nonempty } S} (-1)^{|S|+1} * W / \text{weight}(S).$$

All subset weights are positive and smaller than the prime modulus, so their modular inverses exist.

## 16.2 Correctness proof

The tail-sum identity counts each outcome once for every time step before completion. At a fixed time, inclusion-exclusion exactly computes the union of missing-type events. Independence of draws gives the power for avoiding a fixed set, and the geometric-series identity sums that probability through all times. Every transformation preserves the same expectation, proving the formula.

## 16.3 Complexity and tests

Subset sums can be built from the least significant bit. There are  $2^{n-1}$  terms, each using one modular inverse, for  $O(2^n \log \text{MOD})$  time and  $O(2^n)$  memory.

```
one coupon -> expectation 1
two equal coupons -> 3
highly unequal weights
verify small cases with a mask-state expectation recurrence
```

## 16.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
const int64 MOD = 1'000'000'007;

int64 power(int64 base, int64 exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1)
            result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int64> weight(n);
    int64 total = 0;
    for (int64& value : weight) {
        cin >> value;
        total += value;
    }

    int subset_count = 1 << n;
    vector<int64> subset_sum(subset_count);
    int64 answer = 0;
    for (int mask = 1; mask < subset_count; ++mask) {
        int bit = __builtin_ctz(mask);
        subset_sum[mask] = subset_sum[mask ^ (1 << bit)] + weight[bit];
        int64 term = total * power(subset_sum[mask], MOD - 2) % MOD;
        if (__builtin_popcount(static_cast<unsigned>(mask)) & 1)
            answer = (answer + term) % MOD;
        else
            answer = (answer - term + MOD) % MOD;
    }

    cout << answer << '\n';
}
```

## 16.5 Full Python solution

```
import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    weight = data[1:]
    total = sum(weight)

    subset_sum = [0] * (1 << n)
    answer = 0
    for mask in range(1, 1 << n):
        least_bit = mask & -mask
        bit = least_bit.bit_length() - 1
        subset_sum[mask] = subset_sum[mask ^ least_bit] + weight[bit]
        term = total * pow(subset_sum[mask], MOD - 2, MOD) % MOD
        answer += term if mask.bit_count() & 1 else -term

    print(answer % MOD)

if __name__ == "__main__":
    main()
```

# 17 L. Random Divisor Descent

## 17.1 Expectation recurrence

Let  $E[x]$  be the expected remaining replacements from value  $x$ . The base is  $E[1]=0$ . If  $D(x)$  is the set of proper positive divisors of  $x$ , conditioning on the first uniformly chosen divisor gives:

$$E[x] = 1 + \sum_{d \in D(x)} E[d] / |D(x)|.$$

Every proper divisor is smaller than  $x$ , so increasing numeric order is a valid DP order. Enumerating all divisors separately for every query would repeat work. Instead, when  $E[d]$  becomes known, contribute it to every strict multiple  $2d, 3d, \dots$ . Maintain for each future value both the number of proper divisors seen and the sum of their expectations.

## 17.2 Correctness proof

For  $x=1$ , zero steps are required. Assume expectations are correct below  $x$ . Its possible first moves are exactly its proper divisors, all below  $x$  and chosen with equal probability. The law of total expectation therefore gives the stated recurrence, using already-correct states. The divisor sieve adds  $E[d]$  once to a number precisely when  $d$  is its proper divisor, so the maintained count and sum equal the recurrence quantities. Induction proves all answers.

## 17.3 Complexity and tests

The strict-multiple loops perform  $O(M \log M)$  total iterations, with  $O(M)$  memory and  $O(1)$  output work per query. Divisor counts are below the prime, so their inverses exist.

```
x=1; a prime x (only divisor 1); a prime square
x=4 -> 1 + (E[1]+E[2])/2 = 3/2
a number with many proper divisors
do not include x itself, which would create a self-reference
```

## 17.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
const int64 MOD = 1'000'000'007;
```



```

int64 power(int64 base, int64 exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1)
            result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    vector<int> queries(q);
    int maximum = 1;
    for (int& value : queries) {
        cin >> value;
        maximum = max(maximum, value);
    }

    vector<int> divisor_count(maximum + 1);
    vector<int64> expectation_sum(maximum + 1);
    vector<int64> expectation(maximum + 1);

    for (int divisor = 1; divisor <= maximum; ++divisor) {
        if (divisor > 1) {
            expectation[divisor] = (
                1 + expectation_sum[divisor]
                * power(divisor_count[divisor], MOD - 2)
            ) % MOD;
        }
        for (int multiple = 2 * divisor;
            multiple <= maximum;
            multiple += divisor) {
            ++divisor_count[multiple];
            expectation_sum[multiple] += expectation[divisor];
            expectation_sum[multiple] %= MOD;
        }
    }

    for (int value : queries)
        cout << expectation[value] << '\n';
}

```

## 17.5 Full Python solution

```
import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    queries = data[1:]
    maximum = max(queries)

    divisor_count = [0] * (maximum + 1)
    expectation_sum = [0] * (maximum + 1)
    expectation = [0] * (maximum + 1)

    for divisor in range(1, maximum + 1):
        if divisor > 1:
            expectation[divisor] = (
                1
                + expectation_sum[divisor]
                * pow(divisor_count[divisor], MOD - 2, MOD)
            ) % MOD

        for multiple in range(2 * divisor, maximum + 1, divisor):
            divisor_count[multiple] += 1
            expectation_sum[multiple] += expectation[divisor]
            expectation_sum[multiple] %= MOD

    print("\n".join(str(expectation[value]) for value in queries))

if __name__ == "__main__":
    main()
```